



**SENAC RIO GRANDE DO SUL
TÉCNICO EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**

**GABRIELLY PEREIRA MACHADO
YASMIN NICOLE CAVALHEIRO DA SILVA**

PETCARE: SISTEMA DE GESTÃO PARA PET SHOP

**MONTENEGRO
2026**

CRONOGRAMA DAS ATIVIDADES REALIZADAS

17/08 - Criação base do cronograma de tarefas que devem ser realizadas.

17/08 - Começamos criando um banco novo no PgAdmin e logo começamos a criação da tabela cliente. (Gabrielly)

Query Query History

```
1  CREATE TABLE cliente (  
2      id_cliente SERIAL PRIMARY KEY,  
3      nome VARCHAR(200) NOT NULL,  
4      cpf VARCHAR(14),  
5      telefone VARCHAR(20) NOT NULL,  
6      email VARCHAR(100),  
7      endereco VARCHAR(250) NOT NULL  
8  );
```

17/08 - Em seguida, criamos a tabela "funcionário", onde será o começo para a criação da classe "funcionário" no VsCode. (Yasmin)

```
1  CREATE TABLE funcionario (  
2      id_funcionario SERIAL PRIMARY KEY,  
3      nome VARCHAR(200) NOT NULL,  
4      cpf VARCHAR(14) NOT NULL,  
5      telefone VARCHAR(20) NOT NULL,  
6      cargo VARCHAR(100) NOT NULL  
7  );
```

17/08 - Yasmin ficou responsável pelas tabelas no banco de dados enquanto Gabrielly criava os códigos no VisualCode do Cliente e ClienteDAO (o código está sendo feito com comentários explicando cada parte para ficar visualmente mais explicativo)

24/08 - Foi criada as demais classes, como funcionário, vendas, pets, produto no banco de dados pela Yasmin, depois Gabrielly ficou responsável pelos códigos no VisualCode das classes e seus respectivos DAO's

31/08 - Após um erro de entendimento, tivemos que fazer alguns ajustes nos nossos códigos e banco, juntamente criando novas classes e tabelas para melhor desenvolvimento do projeto (Gabrielly e Yasmin. Começamos o desenvolvimento das novas classes e seus DAO's, cada integrante ficou responsável por fazer uma parte do projeto. Também enviamos as classes que já estavam prontas para nosso repositório no GitHub

02/09 - Continuamos o desenvolvimento das nossas classes e iniciamos a correção do código anterior para a versão com interface.

09/09 - Iniciamos a finalização do projeto, terminando as classes que faltavam para nosso código, desenvolvendo nossa tela e mandando nosso banco de dados

21/09 - Terminamos o desenvolvimento do nosso projeto, buscamos ajuda com a inteligência artificial para melhor visualização na parte da interface.

CREATE TABLES:

ATENDIMENTO

```
-- Table: public.atendimento

-- DROP TABLE IF EXISTS public.atendimento;

CREATE TABLE IF NOT EXISTS public.atendimento
(
    id_atendimento integer NOT NULL DEFAULT
nextval('atendimento_id_atendimento_seq'::regclass),
    id_pet integer NOT NULL,
    id_funcionario integer NOT NULL,
    tipo_atendimento character varying(100) COLLATE pg_catalog."default" NOT NULL,
    valor numeric(12,4) NOT NULL,
    data_atendimento timestamp without time zone NOT NULL DEFAULT
CURRENT_TIMESTAMP,
    CONSTRAINT atendimento_pkey PRIMARY KEY (id_atendimento),
    CONSTRAINT fk_atendimento_funcionario FOREIGN KEY (id_funcionario)
REFERENCES public.funcionario (id_funcionario) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION,
    CONSTRAINT fk_atendimento_pet FOREIGN KEY (id_pet)
REFERENCES public.pet (id_pet) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.atendimento
OWNER to postgres;
```

CLIENTE

```
-- Table: public.cliente

-- DROP TABLE IF EXISTS public.cliente;

CREATE TABLE IF NOT EXISTS public.cliente
(
    id_cliente integer NOT NULL DEFAULT nextval('cliente_id_cliente_seq'::regclass),
    nome character varying(200) COLLATE pg_catalog."default" NOT NULL,
    cpf character varying(14) COLLATE pg_catalog."default",
```

```
telefone character varying(20) COLLATE pg_catalog."default" NOT NULL,  
email character varying(100) COLLATE pg_catalog."default",  
endereco character varying(250) COLLATE pg_catalog."default" NOT NULL,  
CONSTRAINT cliente_pkey PRIMARY KEY (id_cliente)  
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS public.cliente  
OWNER to postgres;
```

ENTRADA ESTOQUE

```
-- Table: public.entrada_estoque
```

```
-- DROP TABLE IF EXISTS public.entrada_estoque;
```

```
CREATE TABLE IF NOT EXISTS public.entrada_estoque  
(  
    id_entrada integer NOT NULL DEFAULT  
nextval('entrada_estoque_id_entrada_seq'::regclass),  
    id_produto integer NOT NULL,  
    id_funcionario integer NOT NULL,  
    quantidade integer NOT NULL,  
    data_entrada timestamp without time zone NOT NULL,  
    CONSTRAINT entrada_estoque_pkey PRIMARY KEY (id_entrada),  
    CONSTRAINT fk_entrada_funcionario FOREIGN KEY (id_funcionario)  
        REFERENCES public.funcionario (id_funcionario) MATCH SIMPLE  
        ON UPDATE NO ACTION  
        ON DELETE NO ACTION,  
    CONSTRAINT fk_entrada_produto FOREIGN KEY (id_produto)  
        REFERENCES public.produto (id_produto) MATCH SIMPLE  
        ON UPDATE NO ACTION  
        ON DELETE NO ACTION  
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS public.entrada_estoque  
OWNER to postgres;
```

FORNECEDOR

```
-- Table: public.fornecedor
```

```
-- DROP TABLE IF EXISTS public.fornecedor;
```

```

CREATE TABLE IF NOT EXISTS public.fornecedor
(
    id_fornecedor integer NOT NULL DEFAULT
nextval('fornecedor_id_fornecedor_seq'::regclass),
    nome character varying(100) COLLATE pg_catalog."default" NOT NULL,
    cnpj character varying(20) COLLATE pg_catalog."default",
    telefone character varying(20) COLLATE pg_catalog."default",
    email character varying(100) COLLATE pg_catalog."default",
    CONSTRAINT fornecedor_pkey PRIMARY KEY (id_fornecedor)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.fornecedor
    OWNER to postgres;

```

FUNCIONÁRIO

```

-- Table: public.funcionario

-- DROP TABLE IF EXISTS public.funcionario;

CREATE TABLE IF NOT EXISTS public.funcionario
(
    id_funcionario integer NOT NULL DEFAULT
nextval('funcionario_id_funcionario_seq'::regclass),
    nome character varying(200) COLLATE pg_catalog."default" NOT NULL,
    cpf character varying(14) COLLATE pg_catalog."default" NOT NULL,
    telefone character varying(20) COLLATE pg_catalog."default" NOT NULL,
    cargo character varying(100) COLLATE pg_catalog."default" NOT NULL,
    CONSTRAINT funcionario_pkey PRIMARY KEY (id_funcionario)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.funcionario
    OWNER to postgres;

```

PET

```

-- Table: public.pet

-- DROP TABLE IF EXISTS public.pet;

CREATE TABLE IF NOT EXISTS public.pet
(

```

```

id_pet integer NOT NULL DEFAULT nextval('pet_id_pet_seq'::regclass),
nome character varying(100) COLLATE pg_catalog."default" NOT NULL,
especie character varying(50) COLLATE pg_catalog."default" NOT NULL,
raca character varying(50) COLLATE pg_catalog."default",
idade integer,
id_cliente integer NOT NULL,
CONSTRAINT pet_pkey PRIMARY KEY (id_pet),
CONSTRAINT pet_id_cliente_fkey FOREIGN KEY (id_cliente)
    REFERENCES public.cliente (id_cliente) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
)

```

```

TABLESPACE pg_default;

```

```

ALTER TABLE IF EXISTS public.pet
    OWNER to postgres;

```

PRODUTO

```

-- Table: public.produto

```

```

-- DROP TABLE IF EXISTS public.produto;

```

```

CREATE TABLE IF NOT EXISTS public.produto
(
    id_produto integer NOT NULL DEFAULT nextval('produto_id_produto_seq'::regclass),
    nome character varying(200) COLLATE pg_catalog."default" NOT NULL,
    categoria character varying(100) COLLATE pg_catalog."default" NOT NULL,
    preco numeric(12,4) NOT NULL,
    quantidade_estoque integer NOT NULL,
    CONSTRAINT produto_pkey PRIMARY KEY (id_produto)
)

```

```

TABLESPACE pg_default;

```

```

ALTER TABLE IF EXISTS public.produto
    OWNER to postgres;

```

VENDA

```

-- Table: public.venda

```

```

-- DROP TABLE IF EXISTS public.venda;

```

```

CREATE TABLE IF NOT EXISTS public.venda
(

```

```

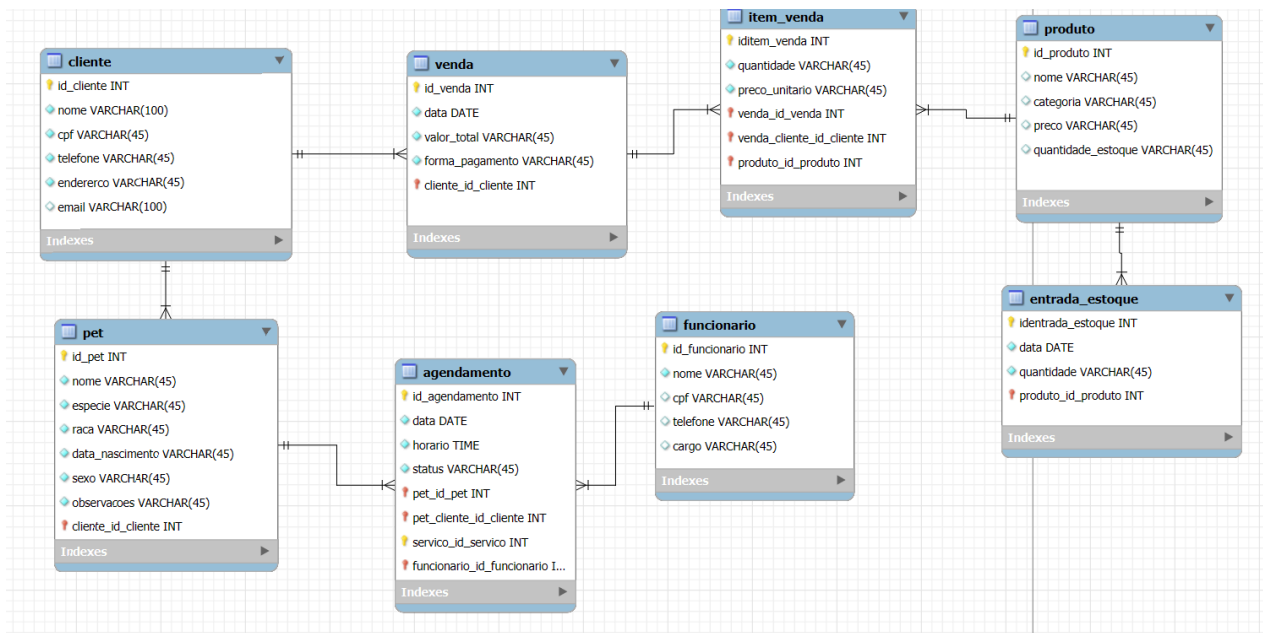
id_venda integer NOT NULL DEFAULT nextval('venda_id_venda_seq'::regclass),
id_cliente integer NOT NULL,
id_produto integer NOT NULL,
quantidade integer NOT NULL,
valor_unitario numeric(12,4) NOT NULL,
data_venda timestamp without time zone NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT venda_pkey PRIMARY KEY (id_venda),
CONSTRAINT fk_venda_cliente FOREIGN KEY (id_cliente)
REFERENCES public.cliente (id_cliente) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION,
CONSTRAINT fk_venda_produto FOREIGN KEY (id_produto)
REFERENCES public.produto (id_produto) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
)

```

TABLESPACE pg_default;

ALTER TABLE IF EXISTS public.venda
OWNER to postgres;

DIAGRAMA INICIAL DO NOSSO PROJETO (DEPOIS FORAM FEITAS ALTERAÇÕES NO PG)



CONCLUSÃO

O projeto PetCare foi desenvolvido começando pela criação do banco de dados no PostgreSQL e, depois, pelas classes e DAO's em Java. Em seguida, foram criadas as interfaces gráficas para facilitar o uso do sistema, incluindo os cadastros, as movimentações e os relatórios.

Durante o desenvolvimento, tivemos algumas dificuldades, principalmente com os relacionamentos entre as tabelas, a integração do Java com o banco de dados e alguns ajustes necessários nas interfaces. No começo, esquecemos que o sistema também precisava de uma interface gráfica, então tivemos bastante dificuldade para adaptar tudo o que já havia sido desenvolvido para as telas. Foi necessário fazer várias alterações no código e realizar novos testes. Durante esse processo, também recorremos algumas vezes à inteligência artificial como apoio para entender alguns erros e melhorar a organização e a visualização da interface.

Depois dos ajustes, conseguimos fazer as principais funcionalidades funcionarem corretamente. Como resultado final, desenvolvemos um sistema para gerenciamento de um Pet Shop, permitindo cadastrar clientes, pets, produtos, funcionários e fornecedores, além de registrar vendas, entradas de estoque e atendimentos. Também foram criados os relatórios e o projeto foi organizado e versionado no GitHub.